SABONG

Gamefowl Management and Record System

Software Design Document

Version 01.20

Revision History

Date	Version	Description	Author
09/05/2026	01.00	SDD Progress part 1	Dumpa, Zachari Abriell F.
10/10/2026	01.10	SDD Progress Update part 2	Borneo, Marbench Gabriel S. Mariquit, Marley Dumpa, Zachari Abriell F. Dionzon, Kirk Dwight
10/23/26	01.20	SDD Progress Update part 3	Dumpa, Zachari Abriell F. Borneo, Marbench Gabriel S.

©Sabong, 2026

Table of Contents

Table of Contents....2

1. INTRODUCTION....3
    1.1 Purpose....3
    1.2 Scope....3
    1.3 Overview....3
    1.4 Reference Material....3
    1.5 Definitions and Acronyms....3
2. SYSTEM OVERVIEW....4
3. SYSTEM ARCHITECTURE....4
    3.1 Architectural Design....4
    3.2 Decomposition Description....5
        3.2.1 User Authentication & Account Subsystem....5
        3.2.2 Chicken Registration & Profile Subsystem....5
        3.2.3 Breeding & Lineage Subsystem....5
        3.2.4 Health & Care Subsystem....5
        3.2.5 Supply & Inventory Subsystem....5
        3.2.6 Event & Match Subsystem....5
        3.2.7 Roster & Record Subsystem....5
        3.2.8 Reporting Subsystem....5
    3.3 Design Rationale....5
4. DATA DESIGN....6
    4.1 Data Description....6
    4.2 Data Dictionary....6
    4.3 Entity Relationship Diagram....9
5. COMPONENT DESIGN....10
    5.1 System Flowchart....10
    5.2 Module Pseudocode....11
        Authentication and Session (SRS 3.1.1, 3.5)....11
        Register Chicken and Dynamic Age (SRS 3.1.2)....11
        Breeding and Pedigree Tree (SRS 3.1.3)....11
        Health Logging, Reminders and Inventory (SRS 3.1.4)....11
        Match, Win Rate and Bracket (SRS 3.1.5)....12
        Reports (SRS 3.1.9)....12
6. HUMAN INTERFACE DESIGN....12
    6.1 Overview of User Interface....12
    6.2 Screen Images....13
    6.3 Screen Objects and Actions....18
7. REQUIREMENTS MATRIX....18
8. APPENDICES....19

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) describes the architecture, data design, component design and user interface of the Gamefowl Management and Record System (GMRS) for Sabong. It translates the requirements in the Software Requirements Specification (SRS, version 01.00) into a design that developers can implement, and serves as the technical reference for developers, system architects and project evaluators.

1.2 Scope

GMRS is a cloud-based web application that replaces manual notebook record-keeping for gamefowl breeders and farm owners. This design covers user authentication and account management, chicken registration and profiles, automated age tracking, breeding and pedigree lineage, health and vaccination logging, care reminders, feed and medicine inventory, match and tournament bracket management, roster and record summary views, and report generation.

1.3 Overview

This document follows the IEEE 1016-2009 structure and is organized into eight sections:

- **Section 1: Introduction –** Purpose, scope, references, definitions and acronyms.
- **Section 2: System Overview –** General description and operational context.
- **Section 3: System Architecture –** Architectural pattern, subsystem decomposition and design rationale.
- **Section 4: Data Design –** Data description and data dictionary.
- **Section 5: Component Design –** System flowchart and procedural logic of the core modules.
- **Section 6: Human Interface Design –** Interface overview, screen layouts, objects and actions.
- **Section 7: Requirements Matrix –** Traceability of SRS requirements to design components.
- **Section 8: Appendices –** Supporting details.

1.4 Reference Material

- Software Requirements Specification (SRS) for Gamefowl Management and Record System, Version 01.00, Sabong, 2026.
- IEEE Std 1016-2009, IEEE Standard for Information Technology – Systems Design – Software Design Descriptions.
- Gamefowl Management UI Mockups and Flowcharts (project documentation).

1.5 Definitions and Acronyms

- **GMRS** - Gamefowl Management and Record System
- **SDD / SRS** - Software Design Document / Software Requirements Specification
- **Sire** - Male parent chicken (broodcock)
- **Dam** - Female parent chicken (broodhen)
- **Offspring** - Chicks produced by a specific breeding pair
- **Pedigree** - Multi-generational lineage tree of a bird
- **ID** - Unique identification number auto-generated for each chicken

- **CRUD** - Create, Read, Update and Delete operations
- **RDBMS** - Relational Database Management System (PostgreSQL / MySQL)
- **ORM** - Object-Relational Mapper
- **MVC** - Model-View-Controller architectural pattern
- **PK / FK** - Primary Key / Foreign Key
- **HTTPS** - HTTP over SSL/TLS

2. SYSTEM OVERVIEW

GMRS addresses the inefficiencies of physical notebooks, whiteboards and disjointed spreadsheets by providing a single, secure, cloud-hosted database for each farm. Breeders need precise records of genetic traits, vaccination timelines, feeding and supplies, and competitive results; without them data is lost, vaccinations are missed, lineage cannot be traced and inventory is mismanaged.

The system automatically calculates each bird's age from its hatch date, visualizes parentage across generations, tracks feed and medicine stock, schedules care reminders, manages tournament match brackets and aggregates win-rate statistics and reports. It is mobile-first so that farm workers can enter data from the field, and it requires an internet connection. Each user can access only the records of their own registered farm.

3. SYSTEM ARCHITECTURE

3.1 Architectural Design

GMRS follows a central-coordinator client-server architecture with five main subsystems: the Gamefowl Owner/Worker Client, the Backend API Server, the AI Recommendation & Analytics Engine, the Weather Connector, and the Database. The Backend API Server acts as the central coordinator: it receives requests from the client, retrieves or stores farm records in the cloud database, queries external weather services, and invokes the AI engine to generate breeding pairings, health alerts, and performance predictions.

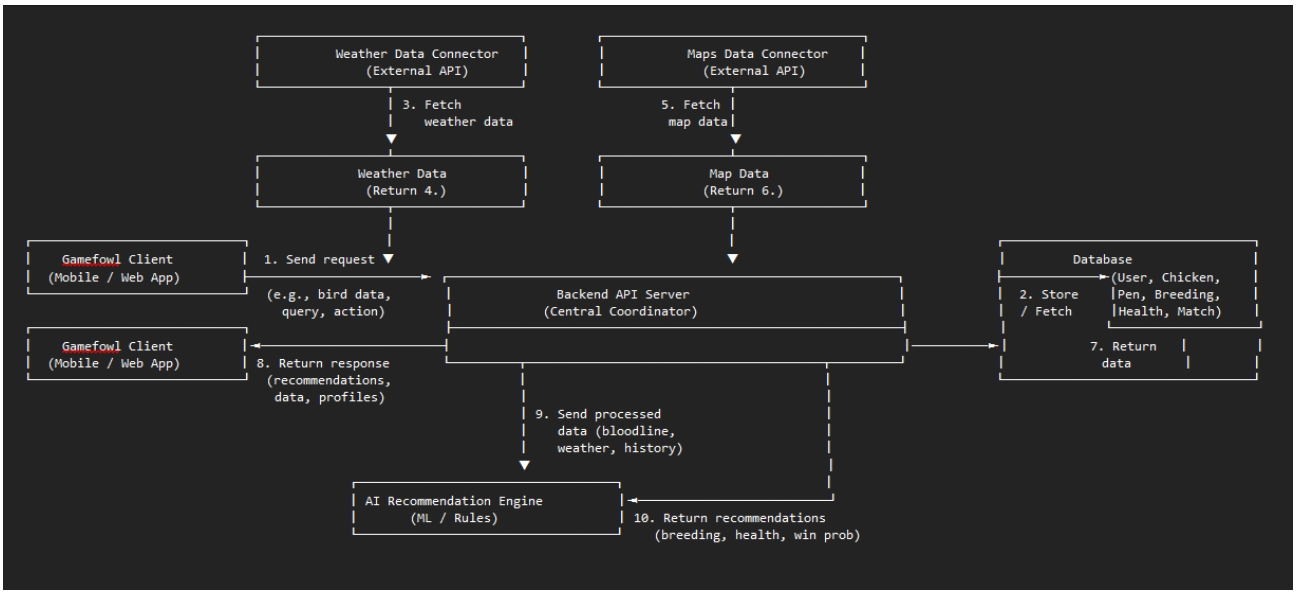


Figure 3.1 – High-level architecture of GMRS

3.2 Decomposition Description

The system is partitioned into the following subsystems:

3.2.1 User Authentication & Account Subsystem

Registration, login/logout, password recovery by email link, session timeout (60 minutes) and farm-level data isolation.

3.2.2 Chicken Registration & Profile Subsystem

Auto-generated unique ID, breed, bloodline, sex, color, photo, status, pen assignment and dynamic age calculation from hatch date.

3.2.3 Breeding & Lineage Subsystem

Records Sire/Dam pairs and mating dates, eggs, hatch rate and linked chicks, and renders the multi-generational pedigree tree.

3.2.4 Health & Care Subsystem

Logs treatments, vaccinations and vitamins per bird or pen, and raises dashboard alerts from care reminders.

3.2.5 Supply & Inventory Subsystem

Tracks feed and medicine stock, deducts usage and shows current balances.

3.2.6 Event & Match Subsystem

Schedules matches, records outcomes (Win, Loss, Draw), updates win-rate statistics and builds tournament brackets.

3.2.7 Roster & Record Subsystem

Roster overview (photo, pen tag, bloodline, age, win-loss), profile setup and detailed record summary.

3.2.8 Reporting Subsystem

Generates chicken, health, breeding, supply and event reports.

3.3 Design Rationale

A modular MVC pattern decouples database entities from the presentation layer, so a change to the mobile or web layout does not disrupt business calculations such as pedigree traversal, age computation or inventory deduction. Storing parentage as relational foreign keys (sire_id and dam_id) lets the lineage tree be assembled in real time without redundant data. An RDBMS was chosen (SRS 3.6.2) because lineage is hierarchical and needs referential integrity, and an ORM (SRS 3.7.2) keeps queries structured and protects data integrity on insert and delete. Every record carries a farm_id so that role-based data isolation (SRS 3.5.1) is enforced in one place, the data-access layer.

4. DATA DESIGN

4.1 Data Description

The information domain of GMRS is managed within a single cloud-based relational database, with data across distinct farms isolated through a unique farm_id. The user entity represents an account holder who owns and accesses a single farm database using an email and a hashed password. Individual birds are recorded as chicken entities, where self-referencing foreign keys for sire and dam establish the multi-generational pedigree. Housing facilities are represented as pen entities, each of which can hold multiple chickens.

Breeding records capture individual mating sessions between a sire and a dam on a specified date, tracking eggs set, hatch results, and offspring links. Treatment, vaccination, vitamin, and routine care logs for an individual bird or an entire pen are stored in health records, while future care events that trigger dashboard alerts are maintained as care reminders. Inventory items, such as feed, medication, or supplements alongside their current quantities, are tracked under supply items. Tournament and sparring events are logged as match records, which specify participating birds, bracket placements, and final outcomes. Computed parameters—such as a bird's dynamic age and overall win rate—are calculated dynamically at read time from stored hatch dates and match results rather than held permanently in storage.

System entity relationships are structured as follows: a user owns multiple chickens, and a single pen houses multiple chickens. A chicken can serve as the sire or dam to multiple offspring chickens. Each breeding record links back to one sire and one dam, while generating multiple offspring. A chicken can accumulate multiple health records and care reminders over time, and participate in multiple match records across its competitive history.

4.2 Data Dictionary

Entities are listed with their attributes, types and descriptions.

Attribute	Data Type	Description
User – Account holder		
user_id	Integer (PK)	Unique user identifier.
email	String, unique	Login email address.
password_hash	String	bcrypt hash of the password (never plain text).
farm_name	String	Registered farm name; scopes all farm data.
Pen – Housing location		
pen_id	Integer (PK)	Unique pen identifier.
farm_id	Integer (FK)	Owning farm.
pen_name	String	Pen number or name (pen tag).
Chicken – Individual gamefowl profile		
chicken_id	String (PK)	Auto-generated unique ID (e.g., GF-001).

farm_id	Integer (FK)	Owning farm.
name	String	Optional name tag.
breed	String	Breed (e.g., Sweater, Kelso).
bloodline	String	Bloodline or lineage line.
sex	Enum	Male, Female.
color	String	Feather pattern or marking.
hatch_date	Date	Used to compute dynamic age.
weight	Float	Latest recorded weight.
status	Enum	Active, Sold, Deceased, Brood.
photo_url	String	Uploaded or camera-captured image.
pen_id	Integer (FK)	Assigned pen; null if unassigned.
sire_id	String (FK)	Father's chicken_id; null if unknown.
dam_id	String (FK)	Mother's chicken_id; null if unknown.
breeding_id	Integer (FK)	Breeding record that produced the bird.
pedigree_notes	Text	Free-text pedigree notes.
BreedingRecord – Pairing and offspring tracking		

Attribute	Data Type	Description
breeding_id	Integer (PK)	Unique breeding session identifier.
sire_id	String (FK)	Father's chicken_id.
dam_id	String (FK)	Mother's chicken_id.
mating_date	Date	Date of pairing.
eggs_count	Integer	Number of eggs laid/set.
hatched_count	Integer	Number hatched; hatch rate = hatched_count / eggs_count.
HealthRecord – Treatment and care log		
record_id	Integer (PK)	Unique health entry identifier.
chicken_id	String (FK)	Target bird (null if pen-wide).
pen_id	Integer (FK)	Target pen (null if single bird).
category	Enum	Vaccination, Vitamin, Medication, Routine Care.
date_administered	Date	Date of administration.
notes	Text	Treatment details or dosage.
CareReminder – Scheduled care event		
reminder_id	Integer (PK)	Unique reminder identifier.
chicken_id	String (FK)	Target bird or pen.
description	String	e.g., "Deworm".
due_date	Date	When the dashboard alert triggers.
is_done	Boolean	Completion flag.
SupplyItem – Feed and medicine inventory		
item_id	Integer (PK)	Unique item code.
farm_id	Integer (FK)	Owning farm.
item_name	String	Feed, vaccine or supplement name.
category	Enum	Feed, Vitamin, Medicine, Equipment.
quantity	Float	Remaining unit quantity (never below 0).
reorder_level	Float	Threshold for low-stock warning.
MatchRecord – Match, event and bracket entry		
match_id	Integer (PK)	Unique match identifier.
event_name	String	Derby or sparring event name.
match_date	Date	Scheduled date.
chicken_a_id	String (FK)	First participant.
chicken_b_id	String (FK)	Second participant.
round_no	Integer	Bracket round (for tournaments).
outcome	Enum	Pending, Win, Loss, Draw (relative to chicken A).
winner_id	String (FK)	Winning bird; null for Draw or Pending.

4.3 Entity Relationship Diagram

The Entity Relationship Diagram (ERD) defines the structural relationships, primary key (PK) and foreign key (FK) links, and cardinalities between the core entities in the GMRS database system.

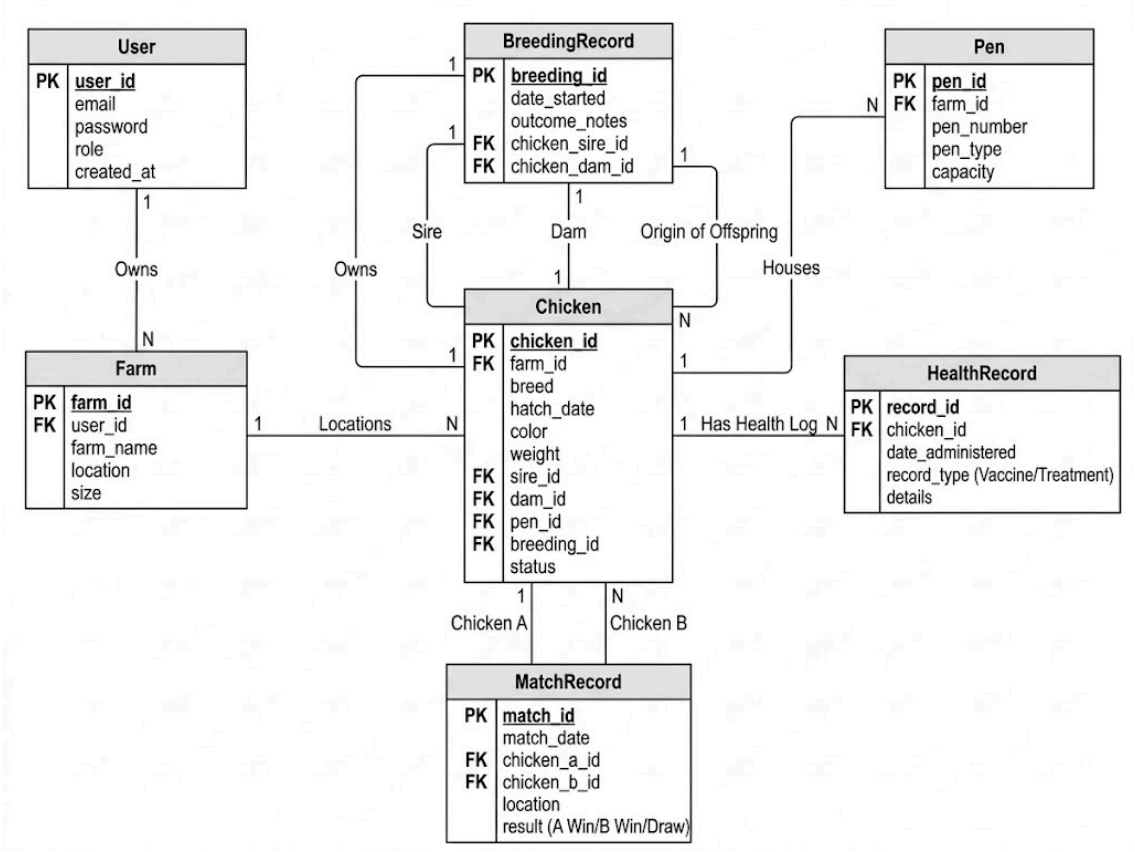


Figure 4.1 – Entity Relationship Diagram for GMRS

Entity Relationship Summary:

- **User to Pen and Chicken (1 to N):** One User owns a farm that contains multiple Pen locations and Chicken profiles (scoped by farm_id).
- **Pen to Chicken (1 to N):** One Pen can house multiple Chicken records.
- **Chicken Pedigree Lineage (1 to N Self-Referencing):** A Chicken can serve as the sire_id (father) or dam_id (mother) to multiple offspring Chicken entities.
- **Chicken to BreedingRecord (1 to N):** A Sire or Dam Chicken can participate in multiple BreedingRecord sessions. Each BreedingRecord links back to one Sire and one Dam, producing multiple offspring.
- **Chicken or Pen to HealthRecord (1 to N):** A Chicken or an entire Pen can accumulate multiple HealthRecord treatment entries over time.
- **Chicken to CareReminder (1 to N):** A Chicken can have multiple scheduled CareReminder alerts.
- **Chicken to MatchRecord (N to M via Foreign Keys):** Each MatchRecord references two participant birds (chicken_a_id and chicken_b_id) and records the optional winner_id.
- **User to SupplyItem (1 to N):** A farm owned by a User tracks multiple SupplyItem inventory records.

5. COMPONENT DESIGN

This section shows the overall operational flow of GMRS and the procedural logic of its core components.

5.1 System Flowchart

After opening the system, a user who is not logged in registers, logs in or recovers a password, then reaches the Farm Central Dashboard. From there the user selects an operation module (Chicken Management, Breeding & Lineage, Health & Supply, or Match & Events). Every module writes to the cloud RDBMS, from which comprehensive reports are generated until the session ends or times out.

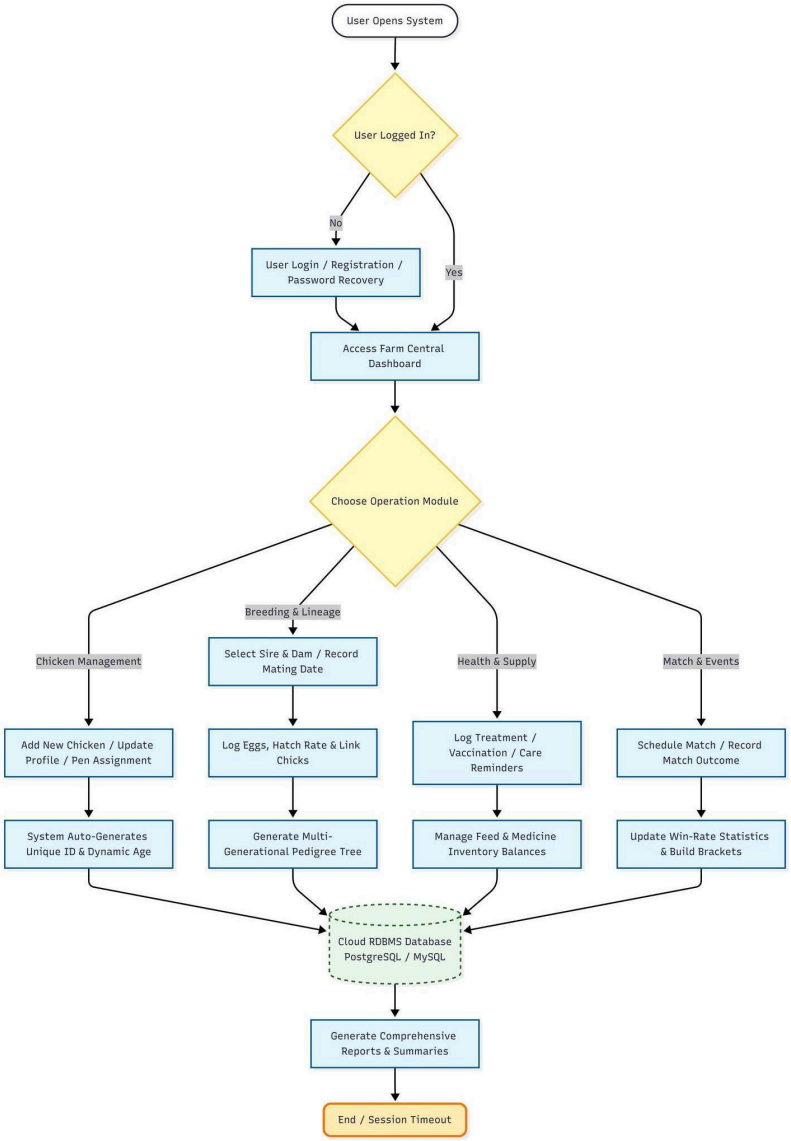


Figure 5.1 – GMRS system flowchart

5.2 Module Pseudocode

Authentication and Session (SRS 3.1.1, 3.5)

```
  FUNCTION login(email,
  password) user = DB.find(User,
  email)
      IF user IS NULL OR NOT bcrypt.verify(password, user.password_hash)
    THEN RETURN error("Invalid email or password")
    session = createSession(user.user_id, timeout = 60 min inactivity)
  RETURN redirect(Dashboard)
```

```
ON every request: IF session.idle > 60 min THEN endSession(); redirect(Login)
```

Register Chicken and Dynamic Age (SRS 3.1.2)

```
FUNCTION registerChicken(form)
      IF any required field (breed, sex, color, hatch_date) is empty THEN
    RETURN highlightMissingFields(red) + plainErrorMessage
    id = nextUniqueId(farm_id)    // e.g., GF-001
  DB.insert(Chicken, id, form, farm_id)
    RETURN profile(id)

FUNCTION computeAge(hatch_date)
    months = monthsBetween(hatch_date, today())
    RETURN (months DIV 12) + " yr " + (months MOD 12) + " mo"
```

Breeding and Pedigree Tree (SRS 3.1.3)

```
FUNCTION recordBreeding(sire_id, dam_id, mating_date)
    ASSERT sire.sex = Male AND dam.sex = Female AND both status IN (Active, Brood)
  DB.insert(BreedingRecord, sire_id, dam_id, mating_date)

  FUNCTION logOffspring(breeding_id, eggs, hatched, chicks[])
  update BreedingRecord(eggs_count, hatched_count)
  FOR each chick: chick.sire_id = sire; chick.dam_id = dam; chick.breeding_id =
breeding_id

FUNCTION buildPedigree(chicken_id, depth)
    IF chicken_id IS NULL OR depth = 0 THEN RETURN
  empty c = DB.get(Chicken, chicken_id)
  RETURN node(c, buildPedigree(c.sire_id, depth-1), buildPedigree(c.dam_id,
depth-1))
```

Health Logging, Reminders and Inventory (SRS 3.1.4)

```
  FUNCTION logTreatment(target, category, date, notes) // target = bird or pen
  FOR each bird IN expand(target): DB.insert(HealthRecord, bird, category, date,
notes)

FUNCTION dashboardAlerts()
  RETURN DB.query(CareReminder WHERE is_done = false AND due_date <= today() + 3
days)

  FUNCTION useSupply(item_id, amount)
  item = DB.get(SupplyItem, item_id)
    IF amount > item.quantity THEN RETURN error("Insufficient stock")
  item.quantity = item.quantity - amount
    IF item.quantity <= item.reorder_level THEN flagLowStock(item)
```

Match, Win Rate and Bracket (SRS 3.1.5)

```
  FUNCTION recordOutcome(match_id, result)
  match.outcome = result; save
    FOR each participant: updateStats(wins, losses, draws)

  FUNCTION winRate(chicken_id)
  total = wins + losses + draws
    RETURN IF total = 0 THEN 0 ELSE (wins / total) * 100

FUNCTION generateBracket(participants[])
    shuffle(participants); pair consecutive birds into round 1 matches
    WHILE winners remain > 1: pair winners of the previous round into the next
  round DB.insert(MatchRecord for each pairing)
```

Reports (SRS 3.1.9)

```
FUNCTION generateReport(type, filters)  // chicken, health, breeding, supply,
event
    rows = DB.query(type, farm_id = session.farm_id, filters)
  RETURN formatPrintable(rows, summaryTotals(rows))
```

6. HUMAN INTERFACE DESIGN

6.1 Overview of User Interface

The interface is a responsive, mobile-first, card-based dashboard with a primary navigation sidebar (a bottom navigation bar on mobile) and modal pop-ups for quick data entry. Core actions such as adding a bird or logging a treatment are reachable within three clicks from the dashboard, and incomplete forms highlight the missing field in red with a plain-language message. Primary navigation provides:

- **Dashboard:** Overview metrics (Total Birds, Active Matches, Inventory Alerts, Care Reminders).
- **Chicken Directory / Roster:** Filterable grid showing photo, pen tag, bloodline, age and win-loss record, with search by ID, name or breed.
- **Profile & Pedigree View:** Chicken info, record summary, interactive Sire/Dam lineage tree and health logs.
- **Care & Schedule Calendar:** Calendar of scheduled vaccinations and care dates.
- **Supply Tracker:** Stock levels with low-inventory warnings.
- **Matches & Brackets:** Upcoming matches, outcomes and tournament brackets.
- **Reports:** Chicken, health, breeding, supply and event reports.

6.2 Screen Images

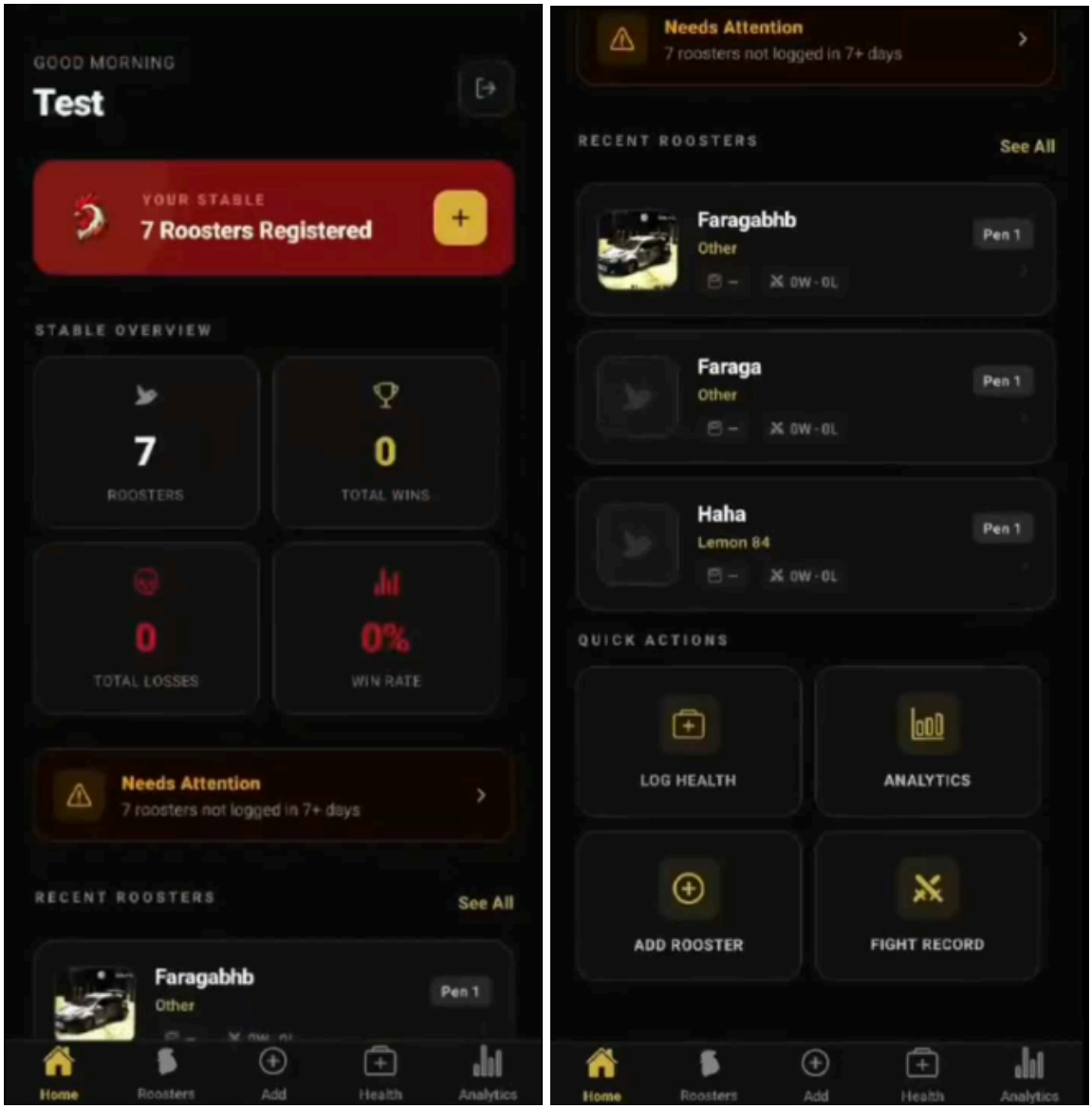


Figure 6.2.1 – Sabong Dashboard Feature

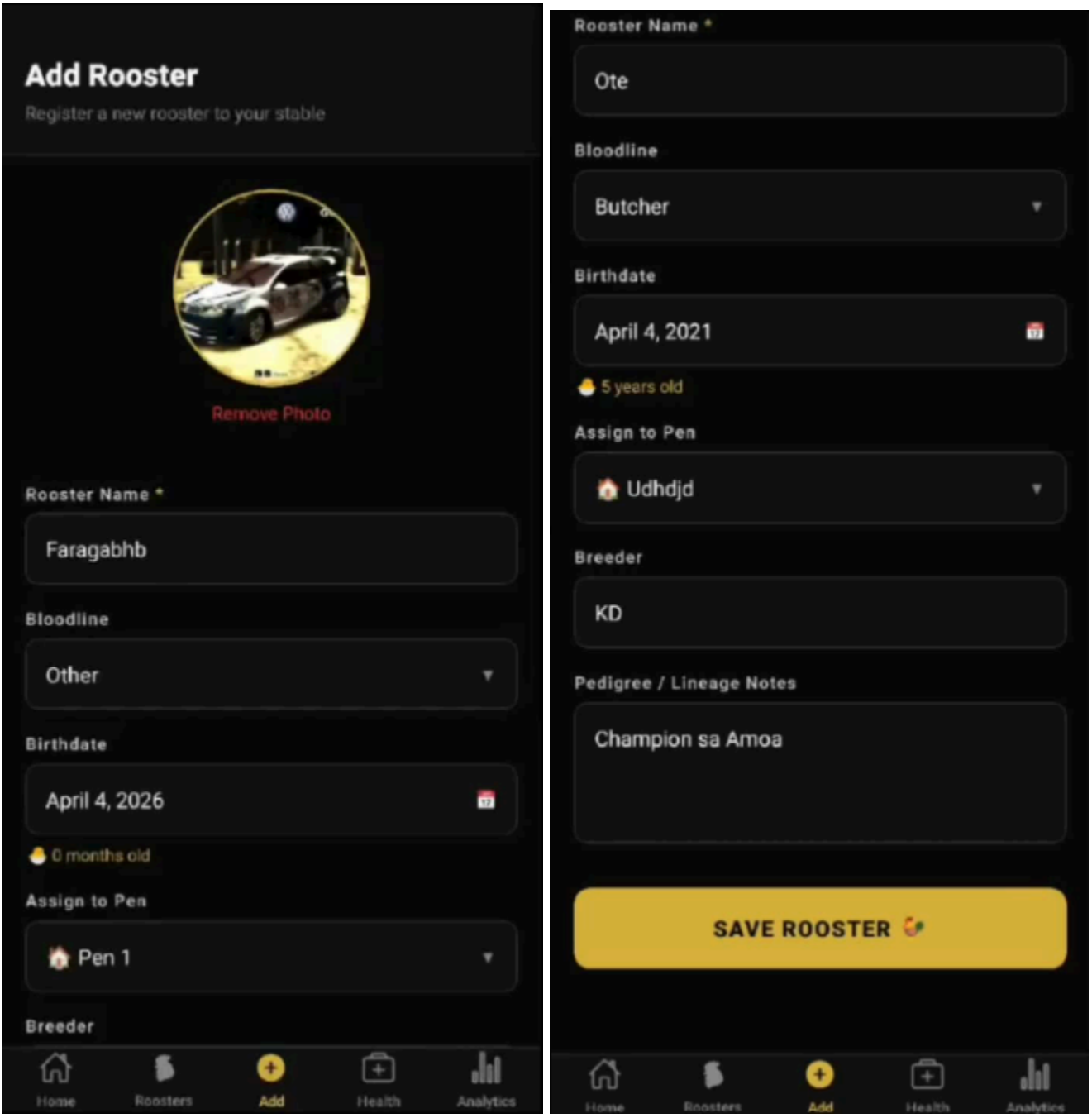


Figure 6.2.2 - Sabong Add Feature

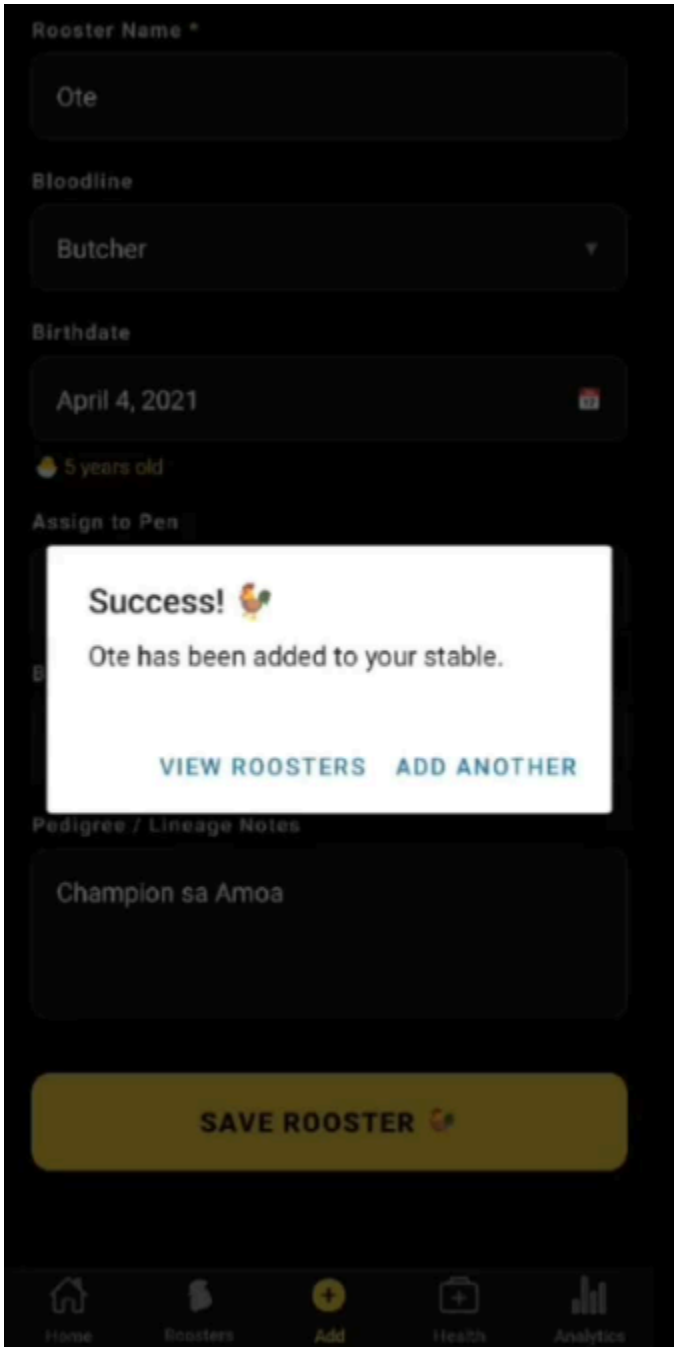


Figure 6.2.3 - Sabong Add Notification

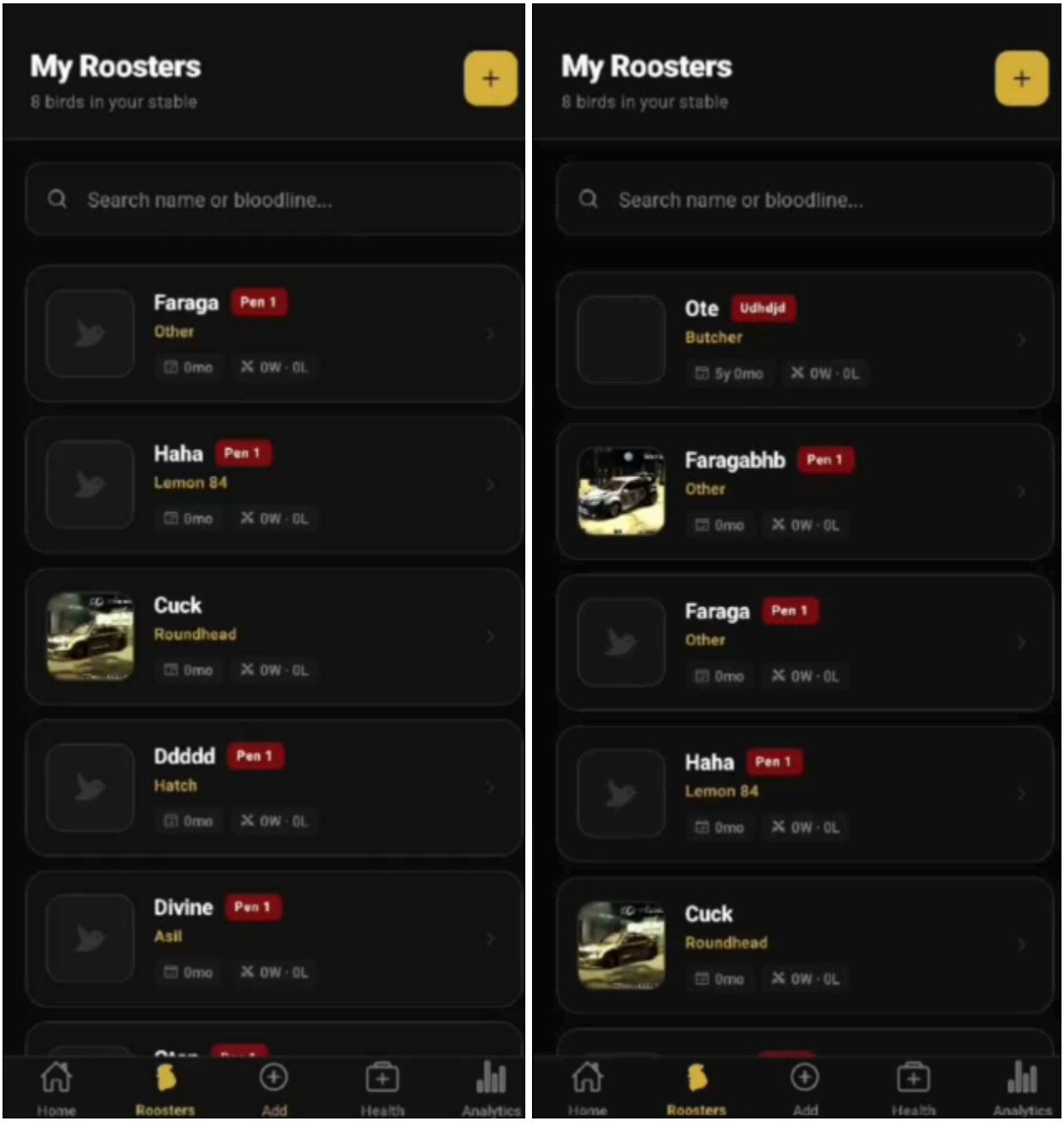


Figure 6.2.4 - Sabong Roosters Feature

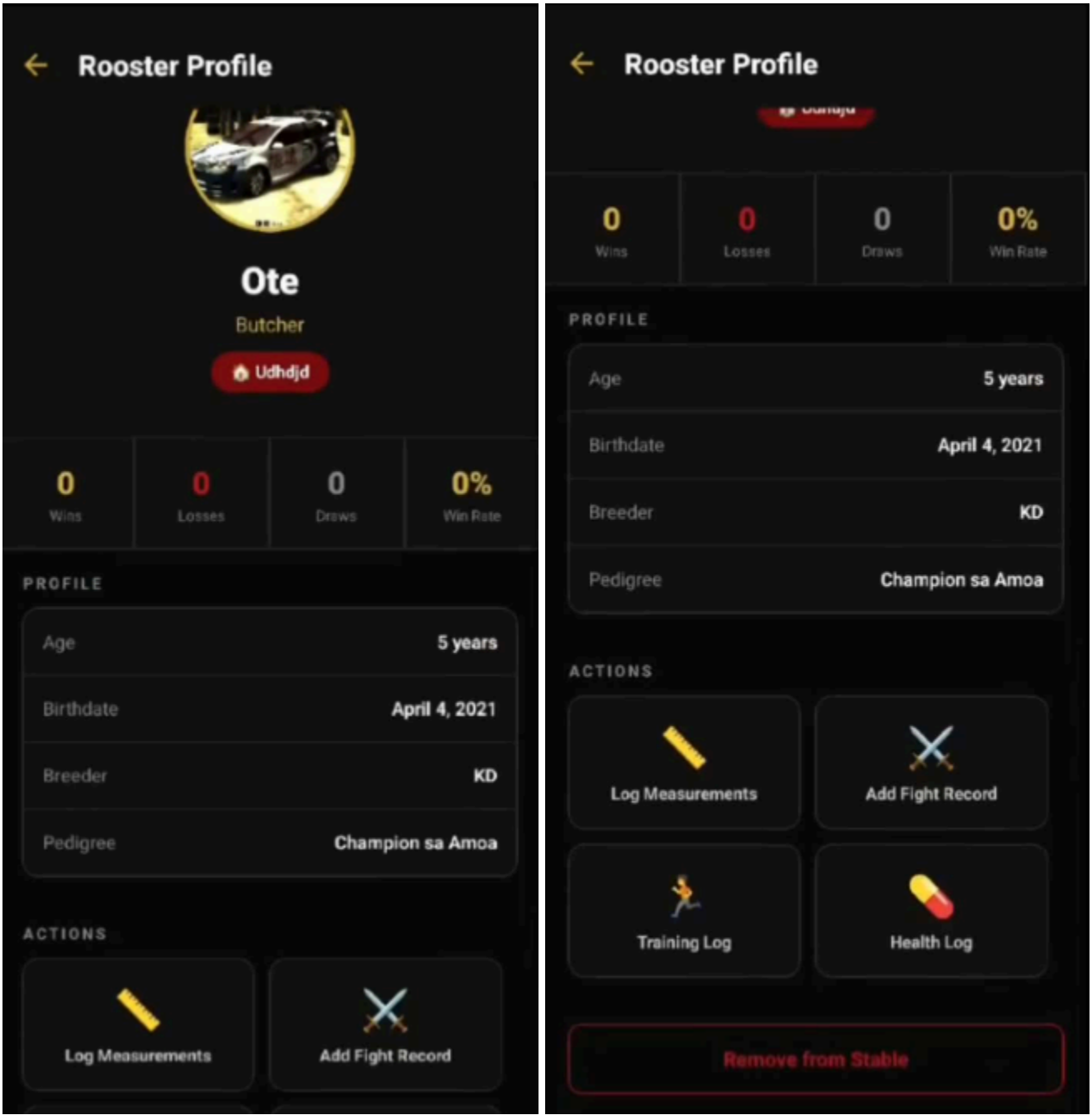


Figure 6.2.5 - Sabong Roosters Profile Feature

6.3 Screen Objects and Actions

Screen Object	Action
[Register Chicken] button	Opens a modal form with breed, bloodline, sex, color, hatch date picker, photo upload/camera capture and optional Sire/Dam selectors; the system assigns the unique ID on save.
[Add Health Log] button	Opens a form to log a vaccination, vitamin, medication or routine care for a bird or pen, and optionally set a care reminder.
[Search Bar] field	Filters the displayed list in real time by chicken_id, name or breed.
[View Pedigree] link	Displays the multi-generational Sire/Dam lineage tree of the selected bird.
[Record Breeding] button	Selects a Sire and Dam from active birds and records the mating date; later logs eggs, hatch rate and links chicks.
[Update Stock] button	Adds stock or deducts quantity used for a feed or medicine item; low stock shows a warning.
[Schedule Match] / [Record Outcome]	Adds a match with assigned birds, and logs Win, Loss or Draw, which updates win-rate statistics.
[Generate Bracket] action	Takes the registered participating birds and builds the tournament bracket, updating it as matches progress.
[Generate Report] button	Produces a printable chicken, health, breeding, supply or event report.

7. REQUIREMENTS MATRIX

The matrix traces SRS functional requirements to the design components and data entities that satisfy them.

SRS Ref.	Requirement	Component / Module	Data Entity
3.1.1	User registration, login/logout, password recovery	Authentication Subsystem	User
3.1.2.1	Add new chicken, auto-generated unique ID	Chicken Registration Module	Chicken
3.1.2.2	Update profile and status	Chicken Registration Module	Chicken
3.1.2.3	Pen assignment	Chicken Registration Module	Pen, Chicken
3.1.2.4	Dynamic age tracking	Age Engine	Chicken (hatch_date)
3.1.3.1	Record breeding pair	Breeding Subsystem	BreedingRecord
3.1.3.2	Track offspring, eggs and hatch rate	Breeding Subsystem	BreedingRecord,

SRS Ref.	Requirement	Component / Module	Data Entity
			Chicken
3.1.3.3	View pedigree tree	Pedigree Generator	Chicken (sire_id, dam_id)
3.1.4.1	Log treatment / vaccination	Health & Care Subsystem	HealthRecord
3.1.4.2	Set care reminders	Health & Care Subsystem	CareReminder
3.1.4.3	Manage supply inventory	Supply Subsystem	SupplyItem
3.1.5.1	Schedule match	Event & Match Subsystem	MatchRecord
3.1.5.2	Record outcomes, win-rate	Event & Match Subsystem	MatchRecord, Chicken
3.1.5.3	Generate bracket	Bracket Controller	MatchRecord
3.1.6	Roster list view	Roster & Record Subsystem	Chicken, Pen, MatchRecord
3.1.7	Profile setup (photo, name, pedigree notes)	Roster & Record Subsystem	Chicken
3.1.8	Record summary	Roster & Record Subsystem	Chicken, MatchRecord
3.1.9	Reports	Reporting Subsystem	All entities
3.1.10	AI constraints and graceful degradation	Application Layer (AI service client); CRUD modules remain independent	Chicken, HealthRecord
3.2	Usability (mobile-first, error handling, three-click rule)	Presentation Layer	—
3.3 / 3.4	Reliability, daily backup, performance, concurrency	Cloud RDBMS and hosting	All entities
3.5	Security: farm-level access, bcrypt, HTTPS, 60-minute timeout	Authentication Subsystem, Data Layer	User
3.6 / 3.7	RDBMS, ORM, browser compatibility, camera interface	Data Layer, Presentation Layer	All entities

8. APPENDICES

- **Database Setup:** Initial PostgreSQL/MySQL migration scripts creating the tables in Section 4.2.
- **Sample Data Population:** Standard list of gamefowl bloodlines (Sweater, Hatch, Kelso, Claret, Roundhead, Asil, Radio, Lemon 84, Grey, Butcher, Albany, Blueface).